Practice 2 ALL — Unified Enterprise Operations, SDP, Delta & Lakeview Suite

Assessment instructions

Difficulty	Marks	Tasks	Working budget
Comprehensive Master	100	10	135 minutes (plus 20 minutes for screenshot preparation)

Objective

Build an end-to-end Azure Databricks solution using a single unified source CSV (`P2_ALL_Master_Enterprise_Dataset.csv`). You will configure Unity Catalog volumes (external and managed), build batch and streaming Spark Declarative Pipelines (SDP) with data quality expectations and SCD Type 1 Change Data Capture (CDC), enforce and evolve Delta schemas, synchronize records via `MERGE`, track row-level modifications with Delta Change Data Feed (CDF) and Time Travel, optimize physical layout with `ZORDER` and verify logical equivalence, apply Unity Catalog column masking, and build an interactive Lakeview Dashboard and AI/BI Genie space.

Key terms

- **Managed vs. External Volume**: A managed volume uses the schema's default storage location; an external volume binds to an explicit ADLS Gen2 URI (`abfss://<container>@<storage>.dfs.core.windows.net/...`).
- **SDP Materialized View vs. Streaming Table**: `@dp.materialized_view` recomputes batch datasets declaratively; `@dp.streaming_table` ingests data incrementally and serves as a target for native `apply_changes` CDC flows.
- **Drop Expectation**: A pipeline quality constraint (`@dp.expect_or_drop`) that filters out invalid records from downstream Silver layers while logging dropped metrics in pipeline history.
- **SCD Type 1 CDC**: Maintains one current row per business key (`entity_id`), sequenced by `change_sequence` regardless of physical file arrival order.
- **Schema Enforcement and Evolution**: Delta Lake blocks writes containing unexpected columns (`extra_grade`) by default, and adds them cleanly when `.option("mergeSchema", "true")` is enabled.

- **Delta CDF & Time Travel**: Change Data Feed (`delta.enableChangeDataFeed = true`) captures `update_preimage`, `update_postimage`, `insert`, and `delete` events across commit versions, while `versionAsOf` queries historical snapshots.
- **Duplicate-Preserving Comparison**: Using `EXCEPT ALL` (or `.exceptAll()`) in both directions (`Current minus Baseline` and `Baseline minus Current`) to prove physical `OPTIMIZE ... ZORDER BY` did not alter logical records.
- **Column Mask**: A reusable SQL UDF attached via `ALTER TABLE ... ALTER COLUMN ... SET MASK` that dynamically masks sensitive strings (`******` + last 4 digits) at query time without altering stored data.

Required environment and object names

Use your existing catalog and reuse `practice_2` for all tasks:

```sql
USE CATALOG `your_existing_catalog`;
CREATE SCHEMA IF NOT EXISTS practice_2;
USE SCHEMA practice_2;
```

Item	Required value
Catalog	Your existing assigned catalog (`<your_catalog>`)
Schema	`practice_2`
External volume	`p2_all_ext_files`
ADLS location	`abfss://<container>@storagename224.dfs.core.windows.net/raw/practice_2/all/files/`
Managed volume	`taskv_yourname`
Input path	`/Volumes/<your_catalog>/practice_2/p2_all_ext_files/input/P2_ALL_Master_Enterprise_Dataset.csv`
Export path	`/Volumes/<your_catalog>/practice_2/taskv_yourname/output/completed_records/`

Item	Required value
Required SDP Pipeline	`p2_all_enterprise_pipeline` (Source notebook: `SDP_Enterprise_Master_yourname`)
SDP Targets	`sdp1_bronze_batch`, `sdp2_silver_clean`, `p2_all_claim_events` (stream), `p2_all_current_entities` (CDC)
Managed Delta Table	`p2_all_operations_ledger`
Column Mask Rule	`p2_all_phone_mask`
Dashboard & Genie	Dashboard `p2_all_performance_dashboard` (Dataset `ledger_paid_detail`) & Genie Space

---

Dataset description

`P2_ALL_Master_Enterprise_Dataset.csv`
CSV with a header, 10 data rows, and 14 columns. Blank fields are missing (`NULL`) values. Spaces around `city_gallery` values are intentional.

Column	Required type	Meaning
`record_id`	`STRING` (Text)	Unique event/transaction row identifier (`REC101` to `REC110`).
`entity_id`	`STRING` (Text)	Business entity/customer/claim key (`ENT01` to `ENT07`) used for joins, MERGE, and CDC.
`centre_zone`	`STRING` (Text)	Operational zone (`NORTH_A`, `SOUTH_B`, `EAST_C`) used for aggregations and `ZORDER`.
`city_gallery`	`STRING` (Text)	Location/gallery name with leading/trailing spaces requiring `TRIM` and `UPPER`.

Column	Required type	Meaning
`contact_phone`	`STRING` (Text)	10-character phone string or `NULL`; preserve in storage and protect via column mask.
`quantity_units`	`INT` (Integer)	Number of units/sessions.
`unit_price_fee`	`DECIMAL(12,2)`	Per-unit fee or rate; may be negative (`REC104`) or missing (`REC106`).
`concession_surcharge`	`DECIMAL(12,2)`	Whole-record adjustment; replace `NULL` with `0.00`.
`is_active`	`BOOLEAN`	Active status flag (`true` or `false`).
`event_status`	`STRING` (Text)	`COMPLETED`, `REJECTED`, `VOID`, or `PENDING`.
`event_date`	`STRING` / `DATE`	Date text in `yyyy-MM-dd` format (read as `STRING` in raw layer, convert to `DATE` in Silver).
`change_sequence`	`INT` (Integer)	Ordering sequence for SCD Type 1 CDC (`highest value wins` per `entity_id`).
`change_time`	`TIMESTAMP`	Event timestamp (`YYYY-MM-DDTHH:MM:SS`), informational in CDC.
`extra_grade`	`STRING` (Text)	Populated only in `REC109` (`GRADE_A`) and `REC110` (`GRADE_B`) for Schema Evolution testing.

Task requirements

Task 1: Prepare External and Managed Unity Catalog Volumes (6 marks)

1.1 Use your existing catalog and the `practice_2` schema.
1.2 Create external volume `p2_all_ext_files` at the assigned ADLS child URI (`CREATE EXTERNAL VOLUME IF NOT EXISTS ... LOCATION 'abfss://...'`).
1.3 Create managed volume `taskv_yourname` (`CREATE VOLUME IF NOT EXISTS ...`).
1.4 Use `dbutils.fs.mkdirs()` to create `/Volumes/<your_catalog>/practice_2/p2_all_ext_files/input/` and `/Volumes/<your_catalog>/practice_2/taskv_yourname/output/`.
1.5 Upload `P2_ALL_Master_Enterprise_Dataset.csv` into `p2_all_ext_files/input/` unchanged and confirm its presence using `dbutils.fs.ls()`.

Expected result: Both volumes exist; `P2_ALL_Master_Enterprise_Dataset.csv` is listed inside the external volume `input` directory.
Screenshots to place under this task: 1

- `P2_ALL_T01_A.png` — **Setup UI**: Volume view showing catalog, `practice_2`, `p2_all_ext_files` (`EXTERNAL` type, child URI, and `input` file) alongside `dbutils.fs.ls()` confirmation.

Task 2: Build Batch SDP Materialized Views with Quality Expectations (12 marks)

Create pipeline notebook `SDP_Enterprise_Master_yourname` using:

```python
from pyspark import pipelines as dp
from pyspark.sql import functions as F
```

(Do not use `.write()`, `.save()`, `.collect()`, `.count()`, or `.display()` inside pipeline functions).

2.1 **Bronze Batch (`sdp1_bronze_batch`)**: Define `@dp.materialized_view` reading `P2_ALL_Master_Enterprise_Dataset.csv` with `.option("header", "true")` and explicit `.schema(...)` (keeping `event_date` as `STRING` and retaining all 10 rows and 14 columns without filtering).
2.2 **Silver Clean (`sdp2_silver_clean`)**: Define `@dp.materialized_view` reading `spark.read.table("sdp1_bronze_batch")` (and performing a self-lookup/join on `entity_id` to verify entity integrity).

2.3 Attach a drop expectation `@dp.expect_or_drop("valid_unit_price", "unit_price_fee IS NOT NULL AND unit_price_fee >= 0")` to drop `REC104` (`-50.00`) and `REC106` (`NULL`).
2.4 Standardize `city_gallery` using `F.upper(F.trim(F.col("city_gallery")))`, convert `event_date` using `F.to_date(F.col("event_date"), "yyyy-MM-dd")`, and replace missing `concession_surcharge` with `0.00` (`F.coalesce`).
2.5 Calculate `gross_amount = (F.col("quantity_units") * F.col("unit_price_fee")).cast("decimal(12,2)")` and `net_amount = (F.col("gross_amount") - F.col("concession_surcharge")).cast("decimal(12,2)")`. Keep all cleaned columns plus `gross_amount` and `net_amount`.

Expected result: Bronze keeps 10 raw rows. Silver drops 2 invalid rows (`REC104`, `REC106`) via expectation and retains 8 cleaned rows (`REC101`, `REC102`, `REC103`, `REC105`, `REC107`, `REC108`, `REC109`, `REC110`).
Screenshots to place under this task: 2

- `P2_ALL_T02_A.png` — **Bronze code**: Complete `@dp.materialized_view` definition for `sdp1_bronze_batch` with explicit schema.
- `P2_ALL_T02_B.png` — **Silver code**: Complete `@dp.materialized_view` definition for `sdp2_silver_clean` showing `@dp.expect_or_drop`, `trim/upper`, `to_date`, `coalesce`, and `net_amount` math.

Task 3: Build Streaming Ingestion & SCD Type 1 CDC in SDP, Execute & Export (14 marks)

In the same SDP notebook `SDP_Enterprise_Master_yourname`:
3.1 Define streaming table `@dp.streaming_table` named `p2_all_claim_events` using `spark.readStream.format("cloudFiles").option("cloudFiles.format", "csv").schema(...)` (or streaming CSV read) on the `input/` directory to ingest all 10 events incrementally.
3.2 Create target streaming table `dp.create_streaming_table("p2_all_current_entities")` and define a native `dp.apply_changes` (or `@dp.append_flow` / CDC flow) from `p2_all_claim_events` keyed by `keys=["entity_id"]`, sequenced by `sequence_by=F.col("change_sequence")`, as `stored_as_scd_type=1`, retaining business columns `entity_id`, `centre_zone`, `city_gallery`, `event_status`, `unit_price_fee`, and `is_active` (retaining `REJECTED` and `VOID` states as current rows if they have the highest sequence).
3.3 Configure pipeline `p2_all_enterprise_pipeline` targeting `<your_catalog>.practice_2` in Triggered mode. Run a **Dry run**, then run a **Full update**.
3.4 In a separate validation notebook, read `sdp2_silver_clean`, filter `event_status =`

'COMPLETED', and export as CSV to /Volumes/<your_catalog>/practice_2/taskv_yourname/output/completed_records/ using .write.mode("overwrite").option("header", "true").csv(...). Read back the exported directory and verify 6 completed rows (REC101, REC102, REC107, REC108, REC109, REC110).

Expected result: Pipeline graph shows all 4 datasets (sdp1_bronze_batch -> sdp2_silver_clean, and p2_all_claim_events -> p2_all_current_entities). Current entities table has 7 unique entity_id rows (ENT01 seq 2, ENT02 seq 3, ENT03 seq 1, ENT04 seq 1, ENT05 seq 1, ENT06 seq 1, ENT07 seq 1). Exported CSV read-back shows 6 rows.
Screenshots to place under this task: 2

- P2_ALL_T03_A.png — **CDC & Export code**: Streaming table + apply_changes CDC definition and separate notebook .write.mode("overwrite").option("header", "true").csv(...) code.
- P2_ALL_T03_B.png — **Pipeline result**: Completed pipeline graph showing all 4 datasets, destination <your_catalog>.practice_2, 10 raw rows, 8 Silver rows (2 dropped), and 7 current CDC entities.

Task 4: Create Managed Delta Ledger, Enforce Schema & Evolve Schema (10 marks)

In your interactive engineering notebook:
4.1 Read P2_ALL_Master_Enterprise_Dataset.csv with declared types. Filter to baseline records record_id <= 'REC108' where unit_price_fee IS NOT NULL AND unit_price_fee >= 0 (REC101, REC102, REC103, REC105, REC107, REC108 — 6 rows) and **exclude** the 14th column extra_grade (keep the first 13 columns, plus calculated net_amount = quantity_units * unit_price_fee - coalesce(concession_surcharge, 0.00) -> 14 columns total).
4.2 Before saving or during table creation, enable Delta Change Data Feed (delta.enableChangeDataFeed = true) on managed Delta table p2_all_operations_ledger (unpartitioned, no liquid clustering) and save the 6 baseline rows.
4.3 Prepare incoming batch record_id IN ('REC109', 'REC110') (2 rows) including all columns **plus** extra_grade (GRADE_A, GRADE_B).
4.4 Attempt .write.format("delta").mode("append").saveAsTable("p2_all_operations_ledger") **without** schema evolution and capture the expected schema-mismatch error identifying extra_grade.
4.5 Re-run the append with .option("mergeSchema", "true") once. Query p2_all_operations_ledger ordered by record_id to show all 8 rows (REC101–REC108 with NULL extra_grade, and REC109–REC110 with GRADE_A and GRADE_B). Record the

latest table commit version after this step as `baseline_version` using `DESCRIBE HISTORY p2_all_operations_ledger`.

Expected result: Append without evolution fails on `extra_grade`. Append with `mergeSchema` succeeds, yielding 8 rows in `p2_all_operations_ledger`. Actual `baseline_version` is recorded.
Screenshots to place under this task: 2

- `P2_ALL_T04_A.png` — **Error output**: Readable Delta schema-enforcement error message identifying `extra_grade` on `p2_all_operations_ledger`.
- `P2_ALL_T04_B.png` — **Evolved output**: All 8 saved rows showing `NULL` for old records, `GRADE_A`/`GRADE_B` for `REC109`/`REC110`, and recorded `baseline_version`.

Task 5: Synchronize Records via MERGE and Apply Targeted DML (Update, Insert, Delete) (12 marks)

Using `p2_all_operations_ledger` (starting from `baseline_version` with 8 rows):
5.1 **MERGE Synchronization**: Construct a 2-row DataFrame (`changes_df`) representing rate/status updates for `record_id = 'REC103'` (`unit_price_fee = 1250.00`, `net_amount = 1230.00`, `event_status = 'COMPLETED'`) and a brand-new record `record_id = 'REC111'` (`ENT08`, `EAST_C`, `HYDERABAD`, `9900001008`, `1`, `400.00`, `0.00`, `true`, `COMPLETED`, `2026-10-20`, `1`, `2026-10-20T14:00:00`, `GRADE_A`, `net_amount = 400.00`). Execute `MERGE INTO p2_all_operations_ledger t USING changes_df s ON t.record_id = s.record_id WHEN MATCHED THEN UPDATE SET t.unit_price_fee = s.unit_price_fee, t.net_amount = s.net_amount, t.event_status = s.event_status WHEN NOT MATCHED THEN INSERT *`.
5.2 **Targeted UPDATE**: Execute an `UPDATE` on `p2_all_operations_ledger` where `record_id = 'REC102'`, setting `unit_price_fee = 1300.00`, `concession_surcharge = 50.00`, and `net_amount = 1250.00`. Record this commit version as `update_version`.
5.3 **Targeted INSERT**: Execute `INSERT INTO p2_all_operations_ledger` adding `REC112` (`ENT09`, `NORTH_A`, `PUNE`, `9900001009`, `2`, `200.00`, `10.00`, `true`, `COMPLETED`, `2026-10-20`, `1`, `2026-10-20T15:00:00`, `GRADE_B`, `net_amount = 390.00`).
5.4 **Targeted DELETE**: Execute `DELETE FROM p2_all_operations_ledger WHERE record_id = 'REC105'` (removing the `VOID` record `REC105` from the operational ledger, or keep track of its deletion version as `delete_version`). Record the commit version as `delete_version`.

Expected result: `MERGE` updates `REC103` and inserts `REC111`. Subsequent DML updates `REC102` (`update_version`), inserts `REC112`, and deletes `REC105` (`delete_version`). Total

current rows: $8 + 1 + 1 - 1 = 9$ rows.
Screenshots to place under this task: 2

- `P2_ALL_T05_A.png` — **MERGE & DML code**: Complete `MERGE INTO` statement, targeted `UPDATE`, `INSERT`, `DELETE`, and version capture variables.
- `P2_ALL_T05_B.png` — **Current ledger output**: All 9 current rows ordered by `record_id` with `delete_version` visible.

Task 6: Inspect Delta Change Data Feed (CDF), Time Travel & Operation History (12 marks)

6.1 **Read CDF**: Using `spark.read.format("delta").option("readChangeFeed", "true").option("startingVersion", update_version).option("endingVersion", delete_version).table("p2_all_operations_ledger")` (or SQL `table_changes('p2_all_operations_ledger', update_version, delete_version)`), display `record_id`, `unit_price_fee`, `net_amount`, `event_status`, `_change_type`, `_commit_version`, and `_commit_timestamp`.
6.2 **Time Travel Comparison**: Query `p2_all_operations_ledger VERSION AS OF <baseline_version>` and compare against the current table:

- Show summary counts and `SUM(net_amount)` for `Baseline` (8 rows, total `net_amount = 7465.00`) vs. `Current` (9 rows, total `net_amount = 7335.00`).
- Display side-by-side/labelled `Baseline` vs `Current` rows for `REC102` (`1200.00` vs `1250.00`) and `REC105` (`1000.00` in Baseline, absent in Current).
  6.3 **Operation History**: Run `DESCRIBE HISTORY p2_all_operations_ledger` bounded between `baseline_version` and `delete_version` showing `version`, `timestamp`, `operation` (`MERGE`, `UPDATE`, `WRITE`/`INSERT`, `DELETE`), and `operationParameters`.

Expected result: CDF displays exactly 4 change rows across `update_version` to `delete_version` (`REC102 update_preimage`, `REC102 update_postimage`, `REC112 insert`, `REC105 delete`). Time travel confirms baseline vs. current metrics.
Screenshots to place under this task: 2

- `P2_ALL_T06_A.png` — **CDF output**: All 4 CDF row images (`update_preimage`, `update_postimage`, `insert`, `delete`) with real commit versions and timestamps.
- `P2_ALL_T06_B.png` — **Time Travel & History output**: Baseline vs. Current summary table, `REC102`/`REC105` comparison, and `DESCRIBE HISTORY` output.

Task 7: Run Delta Layout Optimization (ZORDER) & Prove Duplicate-Preserving Equivalence (10 marks)

7.1 Record the pre-optimization table version (`pre_opt_version` = `delete_version`).
7.2 Run `OPTIMIZE p2_all_operations_ledger ZORDER BY (centre_zone)` once and display the returned metrics (a valid no-op on 9 rows is acceptable).
7.3 Read `pre_opt_version` (`df_before`) and the current post-optimization table (`df_after`). Compare all columns in both directions while preserving duplicate row occurrences using `SELECT COUNT(*) FROM (SELECT * FROM p2_all_operations_ledger EXCEPT ALL SELECT * FROM p2_all_operations_ledger VERSION AS OF <pre_opt_version>)` and the reverse direction (or PySpark `.exceptAll()`).
7.4 Display both difference counts (must be `0` and `0`) alongside one printed sentence stating that `OPTIMIZE ZORDER BY` reorganizes physical Parquet file layout without altering logical business rows.

Expected result: `OPTIMIZE ... ZORDER BY (centre_zone)` succeeds; `Current minus Pre-Opt = 0` and `Pre-Opt minus Current = 0`.
Screenshots to place under this task: 2

- `P2_ALL_T07_A.png` — **Optimization & Comparison code**: `OPTIMIZE ... ZORDER BY (centre_zone)` command, metrics output, and bidirectional `EXCEPT ALL` / `.exceptAll()` comparison code.
- `P2_ALL_T07_B.png` — **Equivalence output**: Both difference counts equal to `0`, current row/amount totals, and the required explanation sentence.

Task 8: Apply Unity Catalog Column Masking on Contact Phone Numbers (8 marks)

8.1 Create reusable SQL function `p2_all_phone_mask(phone_val STRING) RETURNS STRING RETURN CASE WHEN phone_val IS NULL THEN NULL ELSE CONCAT('******', RIGHT(phone_val, 4)) END;`.
8.2 Attach the mask to `p2_all_operations_ledger.contact_phone` using `ALTER TABLE p2_all_operations_ledger ALTER COLUMN contact_phone SET MASK p2_all_phone_mask;`.
8.3 Query `DESCRIBE TABLE EXTENDED p2_all_operations_ledger` (or `information_schema.columns`) to prove `p2_all_phone_mask` is attached to `contact_phone`.
8.4 Query `SELECT record_id, entity_id, contact_phone FROM p2_all_operations_ledger ORDER BY record_id` directly without manual string manipulation in the `SELECT` list.

Expected result: Metadata shows `p2_all_phone_mask` bound to `contact_phone`. Direct query displays `******1001`, `******1002`, `NULL` (for `REC107`), `******1007`, `******1008`, `******1009`.
Screenshots to place under this task: 2

- `P2_ALL_T08_A.png` — **Mask code & Attachment metadata**: `CREATE FUNCTION`, `ALTER TABLE ... SET MASK`, and metadata output proving rule attachment.
- `P2_ALL_T08_B.png` — **Protected output**: Direct query showing all 9 `record_id` rows with masked phones (`******XXXX`) and `NULL`.

Task 9: Execute Comprehensive SQL Validation & Rollup Summaries (6 marks)

In Databricks SQL Editor or SQL cells:
9.1 Validate SDP tables: show `COUNT(*)` for `sdp1_bronze_batch` (10), `sdp2_silver_clean` (8), and `p2_all_current_entities` (7), plus `DESCRIBE TABLE sdp2_silver_clean` confirming `event_date` is `DATE` and `net_amount` is `DECIMAL(12,2)`.
9.2 Query `p2_all_operations_ledger` for active records (`is_active = true` and `event_status = 'COMPLETED'`) to produce:

- **By `centre_zone`**: record count and `SUM(net_amount)` for `NORTH_A` (4 rows / `3620.00`), `SOUTH_B` (3 rows / `3750.00` — note `REC103` was updated to `COMPLETED` in Task 5.1!), and `EAST_C` (2 rows / `875.00`), plus the **Overall** rollup (9 rows / `8245.00` — note: in Task 5.1 `REC103 net_amount` became `1230.00`, `REC102` became `1250.00`, `REC111 400.00`, `REC112 390.00`).
- **By `event_date`**: daily `SUM(net_amount)` for `2026-10-18` (`3470.00`), `2026-10-19` (`2115.00`), and `2026-10-20` (`2660.00`).

Expected result: Exact zone, daily, and overall counts and totals verified in SQL.
Screenshots to place under this task: 1

- `P2_ALL_T09_A.png` — **SQL Summary output**: Zone summary with overall total, daily revenue summary, and schema verification.

Task 10: Build Interactive Lakeview Dashboard & AI/BI Genie Space (10 marks)

10.1 Create an AI/BI Lakeview Dashboard named `p2_all_performance_dashboard` backed by dataset `ledger_paid_detail` (`SELECT * FROM <your_catalog>.practice_2.p2_all_operations_ledger WHERE event_status`

`= 'COMPLETED' AND is_active = true`).
10.2 Add two visualizations:

- **Bar chart**: Total `net_amount` by `centre_zone` (`NORTH_A`: `3620.00`, `SOUTH_B`: `3750.00`, `EAST_C`: `875.00`).
- **Line chart**: Total `net_amount` by `event_date` (`2026-10-18`: `3470.00`, `2026-10-19`: `2115.00`, `2026-10-20`: `2660.00`).
  10.3 Add one shared page-level filter on `centre_zone` affecting both charts. Capture the unfiltered dashboard (`All`), then select `SOUTH_B` in the filter to show the filtered bar (`3750.00`) and daily line (`2026-10-18`: `2480.00`, `2026-10-20`: `1270.00`).
  10.4 Open **AI/BI Genie** (Genie Playground), attach `p2_all_operations_ledger` and `sdp2_silver_clean`, and ask: *"What is the total net_amount and record count for each centre_zone?"* Verify the natural-language response matches your SQL summary.

Expected result: Unfiltered and `SOUTH_B`-filtered Lakeview charts plus Genie Playground natural-language verification match the exact SQL numbers.
Screenshots to place under this task: 2

- `P2_ALL_T10_A.png` — **Unfiltered Dashboard**: Named dashboard `p2_all_performance_dashboard` with both unfiltered charts and shared `centre_zone` **filter set to** `All`.
- `P2_ALL_T10_B.png` — **Filtered Dashboard & Genie**: Dashboard filtered to `SOUTH_B` (`3750.00` bar; `2480.00` and `1270.00` line points) alongside Genie Playground response.

Expected results and evaluation reference

1. Final `p2_all_operations_ledger` State (9 Rows after Task 5 DML)

record_id	entity_id	centre_zone	contact_phone (Masked)	quantity_units	unit_price_fee	concession_surcharge	net_amount	event_status	event_date	extra_grade
REC101	ENT01	NORTH_A	******1001	2	500.00	10.00	990.00	COMPLETED	2026-10-18	NULL
REC102	ENT02	SOUTH_B	******1002	1	1300.00	50.00	1250.00	COMPLETED	2026-10-18	NULL
REC103	ENT02	SOUTH_B	******1002	1	1250.00	20.00	1230.00	COMPLETED	2026-10-18	NULL
REC107	ENT06	EAST_C	NULL	5	100.00	25.00	475.00	COMPLETED	2026-10-19	NULL
REC108	ENT01	NORTH_A	******1001	3	550.00	10.00	1640.00	COMPLETED	2026-10-19	NULL

record_id	entity_id	centre_zone	contact_phone (Masked)	quantity_units	unit_price_fee	concession_surcharge	net_amount	event_status	event_date	extra_grade
REC109	ENT07	NORTH_A	******1007	2	300.00	10.00	590.00	COMPLETED	2026-10-20	GRADE_A
REC110	ENT02	SOUTH_B	******1002	2	650.00	30.00	1270.00	COMPLETED	2026-10-20	GRADE_B
REC111	ENT08	EAST_C	******1008	1	400.00	0.00	400.00	COMPLETED	2026-10-20	GRADE_A
REC112	ENT09	NORTH_A	******1009	2	200.00	10.00	390.00	COMPLETED	2026-10-20	GRADE_B

(Note: At Task 4 `baseline_version` [8 rows: `REC101`–`REC103`, `REC105`, `REC107`–`REC110`], `REC102` had `1200.00`, `REC103` had `1180.00`, `REC105` had `1000.00`, yielding a Baseline total `net_amount` of `7355.00`. After Task 5 MERGE + Update + Insert + Delete, the 9-row Current total `net_amount` is `8245.00`).

2. Zone & Date Aggregations (Task 9 & Task 10)

centre_zone	Records	Total net_amount
NORTH_A	4	3620.00
SOUTH_B	3	3750.00
EAST_C	2	875.00
Overall	**9**	**8245.00**

event_date	All Zones Paid	SOUTH_B Filtered Paid
2026-10-18	3470.00	2480.00
2026-10-19	2115.00	0.00 (or omitted)
2026-10-20	2660.00	1270.00

Reference Implementation Code (All Tasks)

Setup & Task 1 (SQL + Python)

```sql
USE CATALOG `your_existing_catalog`;
```

```
CREATE SCHEMA IF NOT EXISTS practice_2;
USE SCHEMA practice_2;

CREATE EXTERNAL VOLUME IF NOT EXISTS p2_all_ext_files
LOCATION
'abfss://<container>@storagename224.dfs.core.windows.net/raw/practice_2/all/files/
';

CREATE VOLUME IF NOT EXISTS taskv_yourname;
```

Python

```
catalog = "your_existing_catalog"
dbutils.fs.mkdirs(f"/Volumes/{catalog}/practice_2/p2_all_ext_files/input/")
dbutils.fs.mkdirs(f"/Volumes/{catalog}/practice_2/taskv_yourname/output/")
display(dbutils.fs.ls(f"/Volumes/{catalog}/practice_2/p2_all_ext_files/input/"))
```

Task 2 & Task 3A: SDP Pipeline Source (`SDP_Enterprise_Master_yourname.py`)

Python

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType,
    DecimalType, BooleanType, TimestampType
)

catalog = "your_existing_catalog"
input_csv =
f"/Volumes/{catalog}/practice_2/p2_all_ext_files/input/P2_ALL_Master_Enterprise_Da
taset.csv"
input_dir = f"/Volumes/{catalog}/practice_2/p2_all_ext_files/input/"

raw_schema = StructType([
    StructField("record_id", StringType(), True),
    StructField("entity_id", StringType(), True),
    StructField("centre_zone", StringType(), True),
    StructField("city_gallery", StringType(), True),
    StructField("contact_phone", StringType(), True),
    StructField("quantity_units", IntegerType(), True),
    StructField("unit_price_fee", DecimalType(12, 2), True),
    StructField("concession_surcharge", DecimalType(12, 2), True),
    StructField("is_active", BooleanType(), True),
```

```
    StructField("event_status", StringType(), True),
    StructField("event_date", StringType(), True),
    StructField("change_sequence", IntegerType(), True),
    StructField("change_time", TimestampType(), True),
    StructField("extra_grade", StringType(), True)
])

@dp.materialized_view(name="sdp1_bronze_batch")
def sdp1_bronze_batch():
    return (
        spark.read
        .option("header", "true")
        .schema(raw_schema)
        .csv(input_csv)
    )

@dp.materialized_view(name="sdp2_silver_clean")
@dp.expect_or_drop("valid_unit_price", "unit_price_fee IS NOT NULL AND
unit_price_fee >= 0")
def sdp2_silver_clean():
    bronze_df = spark.read.table("sdp1_bronze_batch")
    entity_lookup = bronze_df.select("entity_id").distinct()
    return (
        bronze_df.join(entity_lookup, on="entity_id", how="inner")
        .withColumn("city_gallery", F.upper(F.trim(F.col("city_gallery"))))
        .withColumn("event_date", F.to_date(F.col("event_date"), "yyyy-MM-dd"))
        .withColumn("concession_surcharge",
F.coalesce(F.col("concession_surcharge"), F.lit(0.00)).cast("decimal(12,2)"))
        .withColumn("gross_amount", (F.col("quantity_units") *
F.col("unit_price_fee")).cast("decimal(12,2)"))
        .withColumn("net_amount", (F.col("gross_amount") -
F.col("concession_surcharge")).cast("decimal(12,2)"))
    )

@dp.streaming_table(name="p2_all_claim_events")
def p2_all_claim_events():
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .schema(raw_schema)
        .load(input_dir)
    )

dp.create_streaming_table("p2_all_current_entities")

dp.apply_changes(
```

```
    target="p2_all_current_entities",
    source="p2_all_claim_events",
    keys=["entity_id"],
    sequence_by=F.col("change_sequence"),
    stored_as_scd_type=1,
    column_list=["entity_id", "centre_zone", "city_gallery", "event_status",
"unit_price_fee", "is_active"]
)
```

Task 3B, 4, 5, 6, 7, 8, 9: Engineering & Validation Notebook

Python

```
from pyspark.sql import functions as F

catalog = "your_existing_catalog"
export_path =
f"/Volumes/{catalog}/practice_2/taskv_yourname/output/completed_records/"

# Task 3.4: Export completed Silver records to Managed Volume
(
    spark.read.table(f"{catalog}.practice_2.sdp2_silver_clean")
    .filter(F.col("event_status") == "COMPLETED")
    .write
    .mode("overwrite")
    .option("header", "true")
    .csv(export_path)
)
display(spark.read.option("header", "true").csv(export_path))

# Task 4: Schema Enforcement & Evolution + CDF Enablement
raw_df = (
    spark.read.option("header", "true").schema(raw_schema)

.csv(f"/Volumes/{catalog}/practice_2/p2_all_ext_files/input/P2_ALL_Master_Enterpri
se_Dataset.csv")
    .filter("unit_price_fee IS NOT NULL AND unit_price_fee >= 0")
    .withColumn("city_gallery", F.upper(F.trim(F.col("city_gallery"))))
    .withColumn("event_date", F.to_date(F.col("event_date"), "yyyy-MM-dd"))
    .withColumn("concession_surcharge", F.coalesce(F.col("concession_surcharge"),
F.lit(0.00)).cast("decimal(12,2)"))
    .withColumn("net_amount", (F.col("quantity_units") * F.col("unit_price_fee") -
F.col("concession_surcharge")).cast("decimal(12,2)"))
)

baseline_6_df = raw_df.filter("record_id <= 'REC108'").drop("extra_grade")
incoming_2_df = raw_df.filter("record_id IN ('REC109', 'REC110')")
```

```
spark.sql(f"DROP TABLE IF EXISTS {catalog}.practice_2.p2_all_operations_ledger")
(
    baseline_6_df.write.format("delta")
    .option("delta.enableChangeDataFeed", "true")
    .mode("overwrite")
    .saveAsTable(f"{catalog}.practice_2.p2_all_operations_ledger")
)

# 4.4 Expected failure without mergeSchema
try:

incoming_2_df.write.format("delta").mode("append").saveAsTable(f"{catalog}.practic
e_2.p2_all_operations_ledger")
except Exception as e:
    print("EXPECTED SCHEMA ENFORCEMENT ERROR:", e)

# 4.5 Deliberate schema evolution
(
    incoming_2_df.write.format("delta")
    .option("mergeSchema", "true")
    .mode("append")
    .saveAsTable(f"{catalog}.practice_2.p2_all_operations_ledger")
)

baseline_version = spark.sql(f"DESCRIBE HISTORY
{catalog}.practice_2.p2_all_operations_ledger").selectExpr("max(version) as
v").collect()[0]["v"]
print("Recorded baseline_version:", baseline_version)

# Task 5: MERGE + Update + Insert + Delete
spark.sql(f"""
MERGE INTO {catalog}.practice_2.p2_all_operations_ledger AS t
USING (
    SELECT 'REC103' AS record_id, 'ENT02' AS entity_id, 'SOUTH_B' AS centre_zone,
'BENGALURU' AS city_gallery,
           '9900001002' AS contact_phone, 1 AS quantity_units, CAST(1250.00 AS
DECIMAL(12,2)) AS unit_price_fee,
           CAST(20.00 AS DECIMAL(12,2)) AS concession_surcharge, true AS
is_active, 'COMPLETED' AS event_status,
           DATE'2026-10-18' AS event_date, 2 AS change_sequence,
TIMESTAMP'2026-10-18T09:30:00' AS change_time,
           CAST(1230.00 AS DECIMAL(12,2)) AS net_amount, CAST(NULL AS STRING) AS
extra_grade
    UNION ALL
    SELECT 'REC111', 'ENT08', 'EAST_C', 'HYDERABAD', '9900001008', 1, CAST(400.00
AS DECIMAL(12,2)),
```

```
           CAST(0.00 AS DECIMAL(12,2)), true, 'COMPLETED', DATE'2026-10-20', 1,
TIMESTAMP'2026-10-20T14:00:00',
           CAST(400.00 AS DECIMAL(12,2)), 'GRADE_A'
) AS s
ON t.record_id = s.record_id
WHEN MATCHED THEN UPDATE SET
    t.unit_price_fee = s.unit_price_fee,
    t.net_amount = s.net_amount,
    t.event_status = s.event_status
WHEN NOT MATCHED THEN INSERT *
""")

# 5.2 Targeted Update
spark.sql(f"""
UPDATE {catalog}.practice_2.p2_all_operations_ledger
SET unit_price_fee = 1300.00, concession_surcharge = 50.00, net_amount = 1250.00
WHERE record_id = 'REC102'
""")
update_version = spark.sql(f"DESCRIBE HISTORY
{catalog}.practice_2.p2_all_operations_ledger").selectExpr("max(version) as
v").collect()[0]["v"]

# 5.3 Targeted Insert
spark.sql(f"""
INSERT INTO {catalog}.practice_2.p2_all_operations_ledger
VALUES ('REC112', 'ENT09', 'NORTH_A', 'PUNE', '9900001009', 2, 200.00, 10.00,
true, 'COMPLETED', DATE'2026-10-20', 1, TIMESTAMP'2026-10-20T15:00:00', 390.00,
'GRADE_B')
""")

# 5.4 Targeted Delete
spark.sql(f"DELETE FROM {catalog}.practice_2.p2_all_operations_ledger WHERE
record_id = 'REC105'")
delete_version = spark.sql(f"DESCRIBE HISTORY
{catalog}.practice_2.p2_all_operations_ledger").selectExpr("max(version) as
v").collect()[0]["v"]

# Task 6: CDF & Time Travel
cdf_df = (
    spark.read.format("delta")
    .option("readChangeFeed", "true")
    .option("startingVersion", update_version)
    .option("endingVersion", delete_version)
    .table(f"{catalog}.practice_2.p2_all_operations_ledger")
    .select("record_id", "unit_price_fee", "net_amount", "event_status",
"_change_type", "_commit_version", "_commit_timestamp")
)
display(cdf_df)
```

```
# Task 7: OPTIMIZE ZORDER BY & Duplicate-Preserving Comparison
pre_opt_version = delete_version
display(spark.sql(f"OPTIMIZE {catalog}.practice_2.p2_all_operations_ledger ZORDER
BY (centre_zone)"))

df_pre = spark.read.format("delta").option("versionAsOf",
pre_opt_version).table(f"{catalog}.practice_2.p2_all_operations_ledger")
df_cur = spark.read.table(f"{catalog}.practice_2.p2_all_operations_ledger")
print("Current minus Pre-Opt:", df_cur.exceptAll(df_pre).count())
print("Pre-Opt minus Current:", df_pre.exceptAll(df_cur).count())
print("OPTIMIZE ZORDER BY changes physical file layout without changing logical
business rows.")

# Task 8: Column Masking
spark.sql(f"""
CREATE OR REPLACE FUNCTION {catalog}.practice_2.p2_all_phone_mask(phone_val
STRING)
RETURNS STRING
RETURN CASE WHEN phone_val IS NULL THEN NULL ELSE CONCAT('******',
RIGHT(phone_val, 4)) END
""")
spark.sql(f"""
ALTER TABLE {catalog}.practice_2.p2_all_operations_ledger
ALTER COLUMN contact_phone SET MASK {catalog}.practice_2.p2_all_phone_mask
""")
display(spark.sql(f"DESCRIBE TABLE EXTENDED
{catalog}.practice_2.p2_all_operations_ledger"))
display(spark.sql(f"SELECT record_id, entity_id, contact_phone FROM
{catalog}.practice_2.p2_all_operations_ledger ORDER BY record_id"))
```

Screenshot submission map

Task	Filename	Evidence	Marks
1	`P2_ALL_T01_A.png`	Setup UI (External & Managed Volumes + `dbutils.fs.ls`)	6
2	`P2_ALL_T02_A.png`	Bronze SDP `@dp.materialized_view` code	6

Task	Filename	Evidence	Marks
2	`P2_ALL_T02_B.png`	Silver SDP `@dp.materialized_view` + `@dp.expect_or_drop` code	6
3	`P2_ALL_T03_A.png`	Streaming Table + SCD1 `apply_changes` + CSV Export code	7
3	`P2_ALL_T03_B.png`	Executed SDP Pipeline Graph & Quality/Dropped Metrics	7
4	`P2_ALL_T04_A.png`	Schema Enforcement rejection error on `extra_grade`	5
4	`P2_ALL_T04_B.png`	Schema Evolution (`mergeSchema`) output & `baseline_version`	5
5	`P2_ALL_T05_A.png`	`MERGE INTO` + Targeted `UPDATE`, `INSERT`, `DELETE` code	6
5	`P2_ALL_T05_B.png`	Current 9-row ledger output with `delete_version`	6
6	`P2_ALL_T06_A.png`	Delta CDF output (`update_preimage`, `update_postimage`, `insert`, `delete`)	6
6	`P2_ALL_T06_B.png`	Time Travel (`versionAsOf`) comparison & `DESCRIBE HISTORY`	6
7	`P2_ALL_T07_A.png`	`OPTIMIZE ... ZORDER BY` & bidirectional `EXCEPT ALL` code	5
7	`P2_ALL_T07_B.png`	Zero difference output (`0` and `0`) & layout explanation sentence	5
8	`P2_ALL_T08_A.png`	`p2_all_phone_mask` UDF & `DESCRIBE TABLE EXTENDED` attachment metadata	4

Task	Filename	Evidence	Marks
8	`P2_ALL_T08_B.png`	Direct query output showing masked `******XXXX` and `NULL` phones	4
9	`P2_ALL_T09_A.png`	SQL validation counts, `centre_zone` rollup, and daily totals	6
10	`P2_ALL_T10_A.png`	Unfiltered Lakeview Dashboard `p2_all_performance_dashboard`	5
10	`P2_ALL_T10_B.png`	Filtered Dashboard (`SOUTH_B`) & AI/BI Genie natural-language check	5

Total: 100 marks across 10 tasks and 18 required screenshots.